# DBMS LAB — EXPERIMENT 08

Transactions, COMMIT, ROLLBACK, SAVEPOINT, Isolation Levels and Concurrency Anomalies

Note: These are expected/representative outputs based on the described SQL and sample data, not screenshots from a live MySQL run. Exact formatting, timings and EXPLAIN plans can vary by MySQL version.

## Output 1: Initial balances

SELECT * FROM Bank_Account returns the three starting accounts.

| Account_ID | Account_Holder | Balance |
|---|---|---|
| 101 | Aarav Sharma | 50000.00 |
| 102 | Riya Verma | 30000.00 |
| 103 | Karan Mehta | 20000.00 |

## Output 2: Successful transfer (COMMIT)

5000 is moved from account 101 to 102 and logged. After COMMIT the changes are permanent.

| Account_ID | Account_Holder | Balance |
|---|---|---|
| 101 | Aarav Sharma | 45000.00 |
| 102 | Riya Verma | 35000.00 |
| 103 | Karan Mehta | 20000.00 |

| Log_ID | From_Account | To_Account | Amount | Log_Time |
|---|---|---|---|---|
| 1 | 101 | 102 | 5000.00 | (time of insert) |

## Output 3: Failed transfer (ROLLBACK)

The debit of 7000 from account 101 succeeds inside the transaction, but the credit to account 999 matches 0 rows because that account does not exist (no error is raised). ROLLBACK undoes the debit, so balances are unchanged and no log row is added.

| Account_ID | Account_Holder | Balance |
|---|---|---|
| 101 | Aarav Sharma | 45000.00 |
| 102 | Riya Verma | 35000.00 |
| 103 | Karan Mehta | 20000.00 |

## Output 4: SAVEPOINT for partial failure

101 is debited 3000 and 103 is credited 3000. The deduction of 1000 from account 102 is undone by ROLLBACK TO SAVEPOINT after_credit, and the earlier steps are kept by COMMIT.

| Account_ID | Account_Holder | Balance |
|---|---|---|
| 101 | Aarav Sharma | 42000.00 |
| 102 | Riya Verma | 35000.00 |
| 103 | Karan Mehta | 23000.00 |

## Output 5: Dirty read (READ UNCOMMITTED)

Session 1 adds 10000 to account 101 and does not commit. Session 2, running at READ UNCOMMITTED, sees that uncommitted value, which is a dirty read. If Session 1 later issues ROLLBACK, Session 2 has read data that never existed.

| Session | Statement | Account_ID | Balance seen |
|---|---|---|---|
| Session 1 | SELECT after UPDATE (uncommitted) | 101 | 52000.00 |
| Session 2 | SELECT (READ UNCOMMITTED) | 101 | 52000.00 (dirty read) |

## Output 6: Non-repeatable read (READ COMMITTED)

Session 1 reads account 102, Session 2 adds 2000 and commits, then Session 1 reads the same row again inside the same transaction and gets a different value.

| Step | Session | Balance of account 102 |
|---|---|---|
| First read | Session 1 | 35000.00 |
| UPDATE +2000 and COMMIT | Session 2 | – |
| Second read | Session 1 | 37000.00 (non-repeatable read) |

## Output 7: Phantom read (READ COMMITTED)

Session 2 inserts account 104 (60000) and commits between the two identical COUNT queries in Session 1, so a new "phantom" row appears.

| Step | Session | Accounts_Above_40000 |
|---|---|---|
| First COUNT | Session 1 | 1 |
| INSERT account 104 and COMMIT | Session 2 | – |
| Second COUNT | Session 1 | 2 (phantom row) |

## Output 8: Isolation level comparison

| Statement | Result (Isolation_Level) |
|---|---|
| SELECT @@transaction_isolation (default) | REPEATABLE-READ |
| After SET ... READ UNCOMMITTED | READ-UNCOMMITTED |
| After SET ... READ COMMITTED | READ-COMMITTED |
| After SET ... REPEATABLE READ | REPEATABLE-READ |
| After SET ... SERIALIZABLE | SERIALIZABLE |

Anomalies allowed at each level in MySQL InnoDB (a consistent snapshot read at REPEATABLE READ does not show phantoms):

| Isolation level | Dirty read | Non-repeatable read | Phantom read |
|---|---|---|---|
| READ UNCOMMITTED | Possible | Possible | Possible |
| READ COMMITTED | Prevented | Possible | Possible |
| REPEATABLE READ | Prevented | Prevented | Prevented for snapshot reads |
| SERIALIZABLE | Prevented | Prevented | Prevented |

## Important

These are expected behaviors, not literal server-captured outputs. Row counts, timings, costs and plan wording must be checked against a live run of the exact SQL statements in your repository.